

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 10, Sesión 1 (Sesión 19)

Introducción y Contexto

Resumen de Contenidos Clave

Ejercicios de Repaso

Consejos para la Prueba

Conclusiones y Preparación

Introducción y Contexto

Introducción y Contexto

Objetivos de la Sesión 19

- **Repasar** los conceptos clave para la Solemne II:
 - NumPy: arrays, operaciones vectorizadas, funciones matemáticas
 - NumPy: indexación, máscaras booleanas, estadísticas y matrices
 - Matplotlib: gráficos de línea, scatter, histogramas y mapas de color
 - Python: funciones propias y condicionales simples
- **Resolver** ejercicios integradores similares a los de la evaluación
- **Practicar** código claro, verificable y con interpretación física

Resumen de Contenidos Clave

- Creación de arrays:
 - `np.array()`, `np.linspace()`, `np.arange()`
- Operaciones matemáticas vectorizadas:
 - Operaciones elemento a elemento: `+`, `-`, `*`, `/`, `**`
 - Funciones: `np.sin()`, `np.cos()`, `np.sqrt()`, `np.exp()`
- Funciones estadísticas:
 - `np.mean()`, `np.std()`, `np.min()`, `np.max()`
- Búsqueda de posiciones: `np.argmax()`, `np.argmin()`, `np.where()`

- Indexación directa:
 - `array[0], array[-1], matriz[2, 3]`
- Slicing:
 - `array[inicio:fin], matriz[:, 1], matriz[0, :]`
- Máscaras booleanas:
 - `datos[datos > limite]`
 - `calientes = temperatura > 40`
- **Matrices:** crear con `reshape()` y analizar por filas o columnas.

- Creación de figuras:
 - `plt.figure(figsize=(ancho, alto))`
- Tipos de gráficos:
 - `plt.plot(x, y)` - líneas
 - `plt.scatter(x, y)` - puntos dispersos
 - `plt.hist(datos)` - histogramas
 - `plt.imshow(matriz)` - mapa de color
- Personalización:
 - `plt.xlabel(), plt.ylabel(), plt.title()`
 - `plt.grid(), plt.legend(), plt.colorbar()`
 - `plt.axis("equal")` - ejes proporcionales

Funciones propias

Usar `def` permite separar el modelo físico del resto del análisis.

```
def modelo(x):  
    y = 2*x + 1  
    return y
```

Condicionales

Usar `if/else` permite tomar decisiones según el resultado.

```
if valor > limite:  
    print("supera el limite")  
else:  
    print("dentro del limite")
```

Ejercicios de Repaso



Enunciado

- Una trayectoria parabólica se describe con: $r(\theta) = \frac{p}{1+\cos(\theta)}$
- Datos: $p = 2.0$ UA (parámetro orbital)
- Crear array de ángulos θ de 0 a π con 360 puntos
- Calcular $r(\theta)$ y convertir a coordenadas cartesianas:
 - $x = r \cos(\theta)$
 - $y = r \sin(\theta)$
- Graficar la trayectoria parabólica
- Calcular y mostrar el valor máximo de r con 3 decimales

Conceptos: Arrays NumPy, operaciones vectorizadas, conversión polar-cartesiana, gráficos.

Solución 1 de Referencia: Órbita Parabólica



```
import numpy as np
import matplotlib.pyplot as plt

# Parámetro de la órbita parabólica
p = 2.0 # UA

# 1. Crear array de ángulos theta
theta = np.linspace(0, np.pi, 360)

# 2. Calcular radio r(theta) - ecuación parabólica
r = p / (1 + np.cos(theta))

# 3. Convertir a coordenadas cartesianas
x = r * np.cos(theta)
y = r * np.sin(theta)
```

Ejercicios de Repaso

Solución 1 (continuación): Graficación y Análisis



```
# 4. Graficar la trayectoria parabólica
plt.figure(figsize=(10, 8))
plt.plot(x, y, 'r-', linewidth=2, label='Trayectoria parabólica')
plt.axis('equal')
plt.grid(True, alpha=0.3)
plt.xlabel('x [UA]')
plt.ylabel('y [UA]')
plt.title(f'Trayectoria Parabólica (p={p} UA)')
plt.legend()
plt.show()

# 5. Calcular valor máximo de r
r_max = np.max(r)
print(f"Valor máximo de r: {r_max:.3f} UA")
```

Discusión: La órbita parabólica representa una trayectoria de escape. El valor máximo de r ocurre en $\theta = \pi$.



Enunciado

- Una barra metálica de 10 cm tiene una temperatura variable $T(x)$.
- Crear un array x con 200 puntos entre 0 cm y 10 cm.
- Modelar la temperatura con:
 - $T(x) = 22 + 55 \exp(-x/4) \cos(\pi x/10)$
- Calcular temperatura máxima, mínima y promedio.
- Identificar los puntos donde $T > 40^\circ\text{C}$.
- Escribir una función `temperatura_barra(x)` para calcular $T(x)$.
- Usar un condicional para indicar si la barra supera 70°C .
- Graficar $T(x)$ como línea y destacar los puntos calientes con scatter.

Objetivo: Aplicar arreglos, funciones propias, condicionales, máscaras booleanas y gráficos.



```
import numpy as np
import matplotlib.pyplot as plt

# 1. Definir una función para el modelo
def temperatura_barra(x):
    T = 22 + 55*np.exp(-x/4)*np.cos(np.pi*x/10)
    return T

# 2. Crear posiciones sobre la barra
x = np.linspace(0, 10, 200) # cm

# 3. Calcular temperatura en cada posicion
T = temperatura_barra(x)

# 4. Estadísticas básicas
T_max = np.max(T)
T_min = np.min(T)
T_prom = np.mean(T)

indice_max = np.argmax(T)
x_max = x[indice_max]

print(f"Temperatura maxima: {T_max:.2f} C en x={x_max:.2f} cm")
print(f"Temperatura minima: {T_min:.2f} C")
print(f"Temperatura promedio: {T_prom:.2f} C")

if T_max > 70:
    print("Advertencia: la barra supera 70 C")
else:
    print("La barra no supera 70 C")
```

Ejercicios de Repaso

Solución 2 (continuación): Zonas Calientes



```
# 5. Máscara booleana para puntos calientes
puntos_calientes = T > 40

plt.figure(figsize=(8, 5))
plt.plot(x, T, color="tab:blue", linewidth=2, label="T(x)")
plt.scatter(x[puntos_calientes], T[puntos_calientes],
            color="red", s=25, label="T > 40 C")
plt.scatter(x_max, T_max, color="black", s=80, label="maximo")
plt.axhline(40, color="gray", linestyle="--", label="40 C")

plt.xlabel("Posicion x [cm]")
plt.ylabel("Temperatura [C]")
plt.title("Perfil termico de una barra")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

print(f"Puntos con T > 40 C: {np.sum(puntos_calientes)}")
```

Discusión: La máscara $T > 40$ selecciona solo las posiciones donde la barra supera el umbral.



Enunciado

- Crear un array `t` con 300 valores entre 0 y 12 horas.
- Modelar el brillo con:

$$B(t) = 10 + 0.45 \sin\left(\frac{2\pi t}{4}\right) + 0.12 \cos\left(\frac{2\pi t}{2}\right)$$

- Graficar brillo versus tiempo.
- Calcular brillo máximo, brillo mínimo y amplitud $(B_{\max} - B_{\min})/2$.
- Contar cuántos puntos quedan sobre el brillo promedio.
- Crear una función `brillo_estrella(t)` para calcular $B(t)$.
- Usar un condicional para clasificar la variabilidad como baja o alta.

Conceptos: funciones, condicionales, trigonometría, `max/min`, máscaras booleanas.



```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 12, 300)

def brillo_estrella(t):
    B = 10 + 0.45*np.sin(2*np.pi*t/4)
    B = B + 0.12*np.cos(2*np.pi*t/2)
    return B

B = brillo_estrella(t)

B_max = np.max(B)
B_min = np.min(B)
amplitud = (B_max - B_min) / 2
promedio = np.mean(B)
puntos_sobre_promedio = np.sum(B > promedio)

print(f"Brillo maximo: {B_max:.3f}")
print(f"Brillo minimo: {B_min:.3f}")
print(f"Amplitud: {amplitud:.3f}")
print(f"Puntos sobre el promedio: {puntos_sobre_promedio}")

if amplitud > 0.5:
    print("Variabilidad alta")
else:
    print("Variabilidad baja o moderada")

plt.figure(figsize=(9, 5))
plt.plot(t, B, color="purple", label="brillo")
plt.axhline(promedio, color="black", linestyle="--", label="promedio")
```



```
plt.title("Curva de luz de una estrella variable")  
plt.grid(True, alpha=0.3)  
plt.legend()  
plt.show()
```

Discusión: La señal combina dos oscilaciones. La línea promedio permite distinguir cuándo la estrella está relativamente más brillante.



Enunciado

- Crear un array con 24 temperaturas medidas por sensores.
- Transformarlo en una matriz de 4×6 usando **reshape**.
- Calcular la temperatura media de todo el detector.
- Calcular la media por fila y por columna.
- Encontrar la posición del sensor más caliente.
- Visualizar la matriz con **plt.imshow()** y **plt.colorbar()**.

Conceptos: matrices, **reshape**, **axis**, **argmax**, mapas de color.



```
import numpy as np
import matplotlib.pyplot as plt

temperaturas = np.array([
    18.2, 18.5, 19.1, 19.4, 19.0, 18.7,
    18.8, 19.3, 20.1, 21.4, 20.3, 19.5,
    18.6, 19.0, 20.4, 23.2, 21.0, 19.8,
    18.1, 18.7, 19.6, 20.2, 19.9, 19.1
])

detector = temperaturas.reshape(4, 6)

media_total = np.mean(detector)
media_filas = np.mean(detector, axis=1)
media_columnas = np.mean(detector, axis=0)

indice_max = np.argmax(detector)
fila_max, columna_max = np.unravel_index(indice_max, detector.shape)

print(f"Media total: {media_total:.2f} C")
print("Media por fila:", media_filas.round(2))
print("Media por columna:", media_columnas.round(2))
```



```
print(f"Sensor mas caliente: fila {fila_max}, columna {columna_max}")
print(f"Temperatura maxima: {detector[fila_max, columna_max]:.2f} C")

plt.figure(figsize=(8, 4))
plt.imshow(detector, cmap="inferno")
plt.colorbar(label="Temperatura [C]")
plt.scatter(columna_max, fila_max, color="cyan", s=120,
            edgecolor="black", label="maximo")
plt.xlabel("Columna")
plt.ylabel("Fila")
plt.title("Mapa termico del detector")
plt.legend()
plt.show()
```

Discusión: `reshape` organiza las mediciones como una imagen simple, útil para detectar zonas calientes.



Enunciado

- Crear un array de longitudes L entre 0.20 m y 1.80 m.
- Calcular el período teórico:

$$T = 2\pi\sqrt{\frac{L}{g}}, \quad g = 9.8 \text{ m/s}^2$$

- Crear mediciones observadas sumando pequeñas diferencias conocidas.
- Graficar T_{obs} versus L con scatter.
- Graficar T_{teo} como línea sobre los mismos ejes.
- Calcular residuos $T_{\text{obs}} - T_{\text{teo}}$ y su promedio absoluto.
- Definir una función `periodo_pendulo(L, g)`.
- Usar un condicional para decidir si el ajuste es aceptable.

Conceptos: `sqrt`, comparación modelo-datos, scatter, línea teórica, residuos.



```
import numpy as np
import matplotlib.pyplot as plt

g = 9.8
L = np.linspace(0.20, 1.80, 9)

def periodo_pendulo(L, g):
    T = 2*np.pi*np.sqrt(L/g)
    return T

T_teo = periodo_pendulo(L, g)

diferencias = np.array([0.03, -0.02, 0.01, 0.04, -0.01,
                        0.02, -0.03, 0.01, 0.03])
T_obs = T_teo + diferencias

residuos = T_obs - T_teo
promedio_abs = np.mean(np.abs(residuos))

print("Longitudes:", L.round(2))
print("Periodo teorico:", T_teo.round(3))
print("Periodo observado:", T_obs.round(3))
print("Residuos:", residuos.round(3))
print(f"Promedio absoluto de residuos: {promedio_abs:.3f} s")

if promedio_abs < 0.05:
    print("Modelo aceptable para estas mediciones")
else:
    print("Revisar mediciones o modelo")
```


Ejercicios de Repaso

Solución 5 (continuación):

Modelo y Datos



```
plt.figure(figsize=(8, 5))
plt.scatter(L, T_obs, color="red", s=80, label="datos observados")
plt.plot(L, T_teo, color="blue", label="modelo teorico")
plt.xlabel("Longitud L [m]")
plt.ylabel("Periodo T [s]")
plt.title("Periodo de un pendulo simple")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

Discusión: El gráfico permite comparar directamente el modelo físico con los datos observados.



Enunciado

- Crear un array de longitudes de onda entre 650 nm y 670 nm.
- Modelar la intensidad como:

$$I(\lambda) = 1 + 4 \exp \left[-\frac{(\lambda - 656.3)^2}{2(1.4)^2} \right]$$

- Graficar intensidad versus longitud de onda.
- Encontrar la longitud de onda donde la intensidad es máxima.
- Calcular cuántos puntos tienen intensidad mayor que la mitad del máximo.
- Marcar el máximo con un punto rojo en el gráfico.
- Crear una función `linea_emision(lam, centro, sigma)`.
- Usar un condicional para verificar si el máximo queda cerca del centro esperado.

Conceptos: `np.exp`, `argmax`, máscaras booleanas, gráfico de línea.



```
import numpy as np
import matplotlib.pyplot as plt

lam = np.linspace(650, 670, 400)
centro = 656.3
sigma = 1.4

def linea_emision(lam, centro, sigma):
    I = 1 + 4*np.exp(-((lam - centro)**2) / (2*sigma**2))
    return I

I = linea_emision(lam, centro, sigma)

indice_max = np.argmax(I)
lam_max = lam[indice_max]
I_max = I[indice_max]

mitad_max = I_max / 2
puntos_sobre_mitad = np.sum(I > mitad_max)

print(f"Longitud de onda del maximo: {lam_max:.2f} nm")
print(f"Intensidad maxima: {I_max:.2f}")
print(f"Puntos sobre la mitad del maximo: {puntos_sobre_mitad}")

if abs(lam_max - centro) < 0.1:
    print("Maximo consistente con el centro esperado")
else:
    print("Maximo desplazado respecto al centro esperado")
```



```
plt.figure(figsize=(8, 5))
plt.plot(lam, I, label="espectro")
plt.scatter(lam_max, I_max, color="red", s=90, label="maximo")
plt.axhline(mitad_max, color="gray", linestyle="--",
            label="mitad del maximo")
plt.xlabel("Longitud de onda [nm]")
plt.ylabel("Intensidad relativa")
plt.title("Linea de emision simulada")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

Discusión: La posición del máximo permite estimar el centro de la línea de emisión.

Consejos para la Prueba

Consejos para la Prueba

Estrategias para la Solemne II

Antes de empezar a codificar

- **Lee** el problema completo antes de escribir código.
- **Identifica** si necesitas arrays, filtros, estadísticas o gráficos.
- **Planifica** los pasos: crear datos, definir funciones, calcular, visualizar, interpretar.

Durante la resolución

- Usa **nombres descriptivos** de variables.
- Encapsula modelos repetidos en una **función** con **def**.
- Usa **condicionales** para reportar decisiones o clasificaciones.
- Incluye **comentarios breves** cuando el cálculo tenga varios pasos.
- **Verifica** dimensiones con **shape** cuando uses matrices.
- **Prueba** el código por partes antes de continuar.

Consejos para la Prueba

Errores Comunes a Evitar

NumPy

- Usar `math.sin()` o `math.cos()` sobre arrays en vez de `np.sin()` o `np.cos()`.
- Confundir el índice de un valor con el valor mismo.
- Olvidar que las máscaras booleanas deben tener el mismo tamaño que el array filtrado.
- Usar `reshape()` con un número total de elementos incompatible.

Matplotlib

- Olvidar `plt.show()` al final.
- Entregar gráficos sin etiquetas de ejes, unidades o título.
- No usar `plt.axis("equal")` en trayectorias 2D.
- Poner muchas curvas sin `legend()`.

Conclusiones y Preparación

Conclusiones y Preparación

Resumen de la Sesión 19

- **Repasamos** los comandos esenciales de:
 - NumPy: arrays, operaciones, estadísticas, filtros y matrices.
 - Matplotlib: líneas, scatter, histogramas y mapas de color.
 - Python: funciones propias con **def** y condicionales **if/else**.
- **Practicamos** con ejercicios tipo evaluación:
 - Trayectorias orbitales y coordenadas polares.
 - Perfiles térmicos y selección de zonas con máscaras booleanas.
 - Curvas de luz, detectores, péndulos y espectros.
- **Identificamos** estrategias y errores comunes.

Conclusiones y Preparación

Preparación para la Solemne II

- **Repasen** la sintaxis de los comandos clave:
 - `np.linspace()`, `np.array()`, `reshape()`, operaciones vectorizadas.
 - `np.mean()`, `np.std()`, `np.argmax()`, máscaras booleanas.
 - `plt.plot()`, `plt.scatter()`, `plt.hist()`, `plt.show()`.
 - `def nombre(...):`, `return`, `if`, `else`.
- **Practiquen** escribir código sin consultar apuntes.
- **Revisen** cómo interpretar enunciados y extraer requisitos.
- **Consulten** dudas finales antes de la evaluación.

Importante

La prueba evaluará la capacidad de aplicar NumPy, Matplotlib, funciones y condicionales en problemas físicos o astronómicos similares a los practicados hoy.

- **NumPy Quickstart** - Referencia rápida.
- **Matplotlib Gallery** - Ejemplos de gráficos.
- Apuntes de clases previas: Semanas 6, 7 y 9.

¡Éxito en la Solemne II!

Confíen en su preparación y apliquen lo aprendido.
¡Nos vemos en la próxima sesión!